

39 — Master Implementation Roadmap

Version 1.0 | Implementation Engineering Plan | September 2026

Turn the CloudShield AI specification suite into a buildable, testable, deployable, and governed product.

Document Control & Implementation Contract

Field	Value
Document	CloudShield AI Master Implementation Roadmap
Identifier	39
Version	1.0
Status	Implementation baseline
Scope	Complete CloudShield AI product, including DAIL as a bounded subsystem
Specification baseline	00–38
Primary cloud	AWS for MVP
MVP architecture	Modular application with independently scalable workers; avoid premature microservices
Canonical infrastructure	Terraform
Canonical persistence	Relational system of record + object storage for large evidence/artifacts + durable queue
AI architecture	Provider-neutral AI abstraction layer; hosted model first, local/open-weight model later
Control principle	AI proposes; deterministic systems validate; governance decides; cloud APIs execute; evidence records; trusted state is earned
Roadmap principle	Build in dependency order, prove each gate, then advance

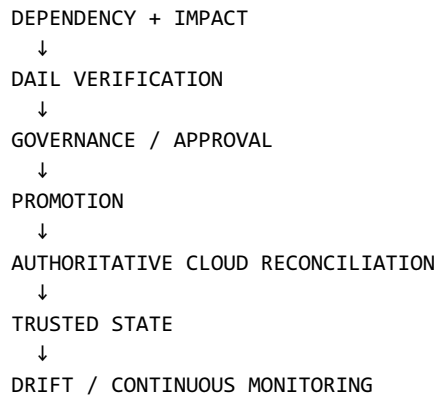
Executive Summary

Specifications 00–38 define what CloudShield AI is, how it behaves, how it is secured, how it is deployed, and how it must perform. This roadmap defines how an engineering team turns that specification baseline into an implemented product. The roadmap is deliberately dependency-driven: foundational identity, tenancy, persistence, authorization, lifecycle, and observability precede higher-risk intelligence and governed cloud mutation.

The MVP is not a collection of disconnected demos. It is a thin but complete vertical slice of the control loop: discover cloud state →normalize assets →analyze security/risk →assemble AI context →produce a structured recommendation →create a remediation candidate →calculate impact →verify through DAIL →obtain governance decision →execute an authorized cloud change where permitted →reconcile authoritative cloud state →record evidence →establish trusted state →monitor drift.

Master Control Loop

```
DISCOVER
↓
INVENTORY
↓
CONFIGURE / OBSERVE
↓
ANALYZE
↓
FINDING + RISK
↓
AI EXPLANATION / RECOMMENDATION
↓
REMEDIATION CANDIDATE
↓
```



1. Roadmap Purpose and Scope

- Convert specifications 00–38 into an ordered engineering execution plan.
- Define workstreams, implementation phases, dependencies, deliverables, gates, acceptance criteria, and ownership boundaries.
- Provide a realistic MVP sequence that avoids premature microservices and unnecessary foundation-model work.
- Make DAIL implementation explicit while preserving its position as a subsystem rather than the whole product.
- Define the minimum complete vertical slice required before broader feature expansion.
- Create a traceable path from specification → implementation task → test → deployment → evidence → production readiness.

This roadmap is an engineering baseline, not a calendar promise. Sprint duration and staffing should be estimated after repository setup and team capacity are known.

2. Implementation Principles

Principle	Implementation rule
Specification-first	Do not code behavior that contradicts 00–38; update the specification through governance when requirements change.
Vertical-slice delivery	Prefer complete end-to-end slices over building every layer in isolation.
Security-first foundations	Tenant isolation, authorization, secret boundaries, audit, and safe defaults are foundational work.
Deterministic core	Identity, lifecycle, policy, verification, state transitions, and evidence must be deterministic.
AI abstraction	No domain module directly depends on a specific LLM vendor SDK.
DAIL as subsystem	DAIL owns verification/decision-support responsibilities and does not become the product boundary.
Async by default	Discovery, analysis, AI-heavy work, verification, evidence generation, and integrations use durable jobs where appropriate.
Fail closed	Critical control paths stop when required safety, verification, authorization, or evidence preconditions are unavailable.
Authoritative confirmation	Promotion is not complete until the relevant

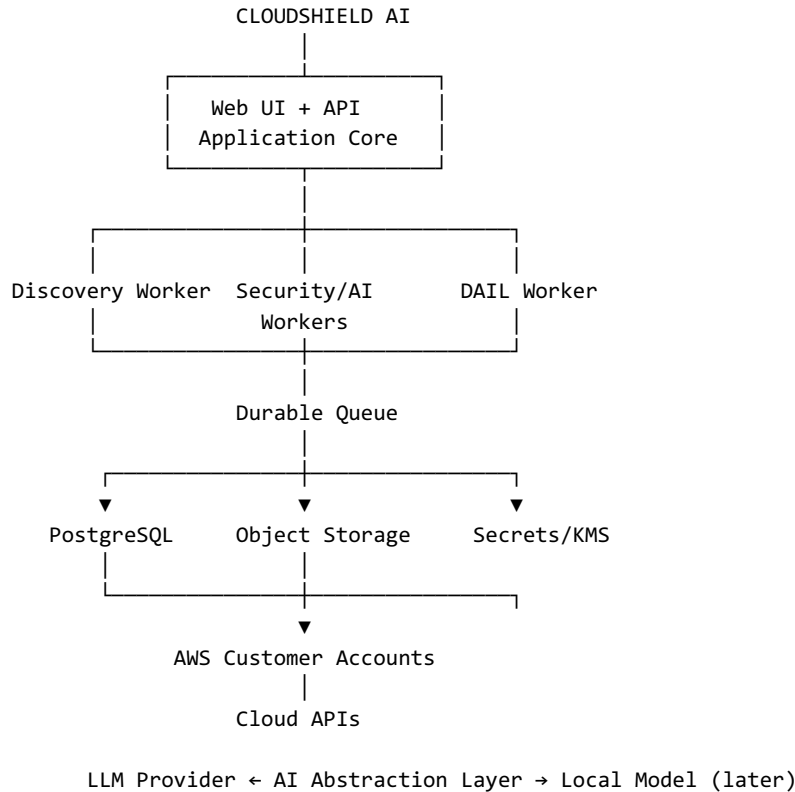
	external/cloud state is reconciled.
Evidence everywhere	Important actions and state transitions produce durable, traceable evidence.
Measured scaling	Performance architecture evolves from observed bottlenecks rather than speculative complexity.
No foundation-model prerequisite	MVP uses an existing model through the abstraction layer; proprietary foundation-model training is future scope.

3. Master Workstream Model

ID	Workstream	Primary responsibility
WS-01	Product / Domain Foundation	Repository, domain primitives, tenancy, identifiers, configuration, core conventions
WS-02	Identity / Authorization	Users, roles, tenant isolation, authorization policy, service identity
WS-03	Persistence / Data	Schema, migrations, repositories, object storage, evidence metadata
WS-04	API / Application	REST API, command/query handling, validation, jobs, lifecycle commands
WS-05	Cloud Integration	AWS connection, STS/IAM role assumption, discovery, observation, reconciliation
WS-06	Security / Risk	Security rules, findings, risk scoring, policy evaluation
WS-07	Dependency / Impact	Canonical dependency graph, identity reconciliation, blast-radius analysis
WS-08	AI / Intelligence	AI gateway, context engineering, structured outputs, evaluation, safety
WS-09	DAIL	Verification, invariants, evidence requirements, promotion readiness
WS-10	Governance / Promotion	Approval, policy gates, promotion orchestration, trusted-state transition
WS-11	Evidence / Audit / Observability	Logs, metrics, traces, domain events, audit, evidence, dashboards
WS-12	Frontend / UX	Application shell, inventory, findings, candidate, verification, governance, trusted state
WS-13	Integrations	Webhooks, ITSM, source control, notifications, external systems
WS-14	Infrastructure / Terraform	AWS infrastructure, environments, IAM, networking, storage, queues, DR
WS-15	DevOps / Release	CI/CD, artifacts, security gates, migrations, deployment, rollback
WS-16	Testing / Quality	Unit, contract, integration, system,

		E2E, performance, security, recovery
WS-17	Security Operations / DR	Detection, incident response, backup/restore, recovery, reconciliation
WS-18	Experimentation / Evaluation	DAIL experiments, AI benchmarks, regression, controlled engineering experiments

4. Target MVP Architecture



The MVP should be deployable as a modular application plus workers. Modules may be separated into services later when measured scale, isolation, release cadence, or security boundaries justify it.

5. Implementation Phases

Phase	Name	Primary outcome	Gate
P0	Engineering Mobilization	Repository, conventions, environments, architecture baseline, backlog	Build-ready
P1	Platform Foundation	Tenant, identity, persistence, API skeleton, jobs, observability	Platform-ready
P2	Cloud Discovery & Inventory	AWS connection, discovery, asset identity, observations	Discovery-ready
P3	Security & Risk Intelligence	Rules, findings, risk, policy	Analysis-ready

		evaluation	
P4	Dependency & Impact	Canonical graph, dependency confidence, impact analysis	Impact-ready
P5	AI Intelligence & Safety	AI gateway, context, structured output, guardrails, evaluation	AI-ready
P6	DAIL Verification	Verification engine, evidence requirements, candidate binding	Verification-ready
P7	Governance & Promotion	Approval, authorized cloud mutation, reconciliation, trusted state	Control-loop-ready
P8	Product UX & Integrations	Complete user workflows, reporting, external integrations	Product-ready
P9	Hardening & Scale	Performance, resilience, security operations, DR, release maturity	Production-ready
P10	MVP Release & Continuous Improvement	Production launch, telemetry-driven iteration, controlled expansion	MVP accepted

P0 — Engineering Mobilization

- Create canonical repository structure and module boundaries.
- Lock language/runtime/toolchain versions and coding standards.
- Translate specifications 00–38 into traceable epics/stories/tasks.
- Create ADR process and change-control mechanism.
- Create local development environment and mock provider/LLM adapters.
- Create CI baseline: formatting, linting, unit tests, dependency/security scanning, secret scanning.
- Create dev/test/staging/prod environment definitions and Terraform skeleton.
- Create engineering Definition-of-Done checklist and evidence conventions.

P1 — Platform Foundation

- Implement tenant/organization/scope primitives and authorization middleware.
- Implement stable IDs, audit actor model, service identity, and permission checks.
- Implement relational schema, migrations, repositories, optimistic concurrency/versioning.
- Implement object-storage abstraction and evidence metadata.
- Implement durable job abstraction, idempotency, retries, dead-letter handling, and job state.
- Implement API envelope, errors, pagination, filtering, correlation IDs, and request validation.
- Implement structured logging, metrics, tracing, health/readiness, and audit pipeline.
- Implement configuration/secrets interfaces and environment-safe defaults.

P2 — Cloud Discovery & Inventory

- Implement AWS provider adapter and customer role-assumption flow.
- Implement CloudConnection lifecycle and secure credential boundaries.
- Implement account/region/service discovery partitions and checkpoints.
- Implement canonical ResourceIdentifier and Asset reconciliation.
- Implement Observation and ConfigurationSnapshot persistence.
- Implement freshness/provenance semantics and stale/partial/unknown states.
- Implement inventory APIs and initial UI.
- Build AWS integration tests using controlled test accounts/mocks.

P3 — Security & Risk Intelligence

- Implement SecurityRule model and deterministic rule execution.
- Implement finding identity, lifecycle, severity, evidence, and remediation metadata.
- Implement risk scoring and material-input invalidation.
- Implement policy evaluation and explainable rule outcomes.
- Implement findings/risk APIs, dashboards, filters, and evidence views.
- Create baseline AWS security controls sufficient for the MVP use cases.
- Add regression datasets and deterministic analysis tests.

P4 — Dependency & Impact

- Implement dependency edge model with source, confidence, timestamps, and provenance.
- Implement dependency discovery from authoritative and derived sources.
- Implement ambiguous identity handling and conflict states.
- Implement graph queries with bounded traversal and depth limits.
- Implement direct/transitive impact analysis bound to exact candidate version.
- Implement impact invalidation on graph/configuration changes.
- Add dependency/impact visualization and APIs.
- Create graph correctness and performance benchmark datasets.

P5 — AI Intelligence & Safety

- Implement provider-neutral AI abstraction layer and provider registry.
- Implement LLM gateway/orchestrator with timeouts, retries, circuit breakers, quotas, and telemetry.
- Implement context assembly using authorized CloudShield state and evidence.
- Implement structured output schemas and domain validation.
- Implement prompt-injection and external-content trust boundaries.
- Implement AI recommendation/explanation lifecycle and uncertainty handling.
- Implement AI evaluation harness using Specification 37.
- Implement safe degradation when AI is unavailable or produces invalid output.

P6 — DAIL Verification

- Implement DAIL identity and verification contracts.
- Implement candidate-version-bound verification.
- Implement deterministic invariants and verification gates.

- Implement evidence requirements and verification evidence production.
- Implement verification lifecycle, retries, timeout handling, and reconciliation.
- Implement verification APIs and operator-facing status views.
- Implement DAIL-specific testing and experiment hooks.
- Ensure ordinary AI/application workloads cannot acquire promotion authority.

P7 — Governance & Promotion

- Implement remediation candidate and version lifecycle.
- Implement READY_FOR_IMPACT →IMPACTED →AWAITING_VERIFICATION →VERIFYING →VERIFIED flow.
- Implement approval and policy decision model.
- Implement promotion preflight checks.
- Implement authorized AWS mutation adapter with least-privilege promotion identity.
- Implement ambiguous mutation reconciliation.
- Implement authoritative post-change observation.
- Implement PROMOTED and TRUSTED state transitions only after all gates.
- Implement explicit failure/block/reject/reconcile states and evidence.

P8 — Product UX & Integrations

- Complete global navigation and application shell.
- Complete cloud connection, inventory, findings, risk, dependency, candidate, verification, approval, and trusted-state experiences.
- Implement jobs, notifications, stale/partial/error/degraded UI states.
- Implement reports and evidence exploration.
- Add initial external integrations such as webhooks/notifications and selected source-control or ITSM workflows where MVP scope permits.
- Ensure external integrations cannot bypass core authorization or lifecycle gates.
- Complete accessibility and UX acceptance testing.

P9 — Hardening & Scale

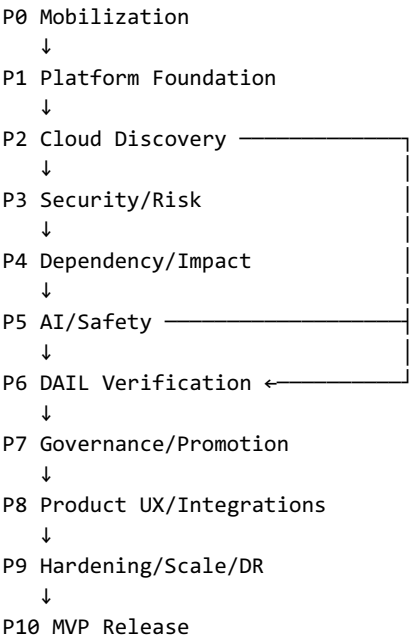
- Execute load, stress, soak, fault-injection, security, tenant-isolation, and DR testing.
- Optimize measured database, worker, queue, cloud, graph, AI, and UI bottlenecks.
- Implement production SLO dashboards and alerts.
- Complete backup/restore drills and reconciliation testing.
- Complete security operations runbooks and incident response integration.
- Validate Terraform drift, deployment rollback, and release recovery.
- Run red-team/adversarial AI and DAIL safety scenarios.
- Close critical and high-risk production-readiness gaps.

P10 — MVP Release & Continuous Improvement

- Freeze MVP scope and create release candidate.
- Run full regression and end-to-end control-loop verification.
- Run production-readiness gate with explicit sign-offs.
- Deploy through governed CI/CD.

- Monitor SLOs, security events, AI quality, cloud failures, reconciliation backlog, and trusted-state drift.
- Operate a controlled post-launch backlog driven by evidence.
- Prioritize reliability, correctness, safety, and measured customer value before broad feature expansion.

6. Master Dependency Graph



Cross-cutting from P1 onward:

Security • Observability • Testing • Terraform • CI/CD • Evidence • Documentation

Dependency	Reason
Identity →Discovery	Assets require stable tenant/scope/provider identity.
Persistence →Lifecycle	State transitions require transactional storage and versioning.
Discovery →Security	Security analysis requires authoritative observations.
Discovery + Identity →Dependency	Graph edges require stable asset identity.
Dependency →Impact	Blast radius requires canonical relationships.
Context + Domain State →AI	AI recommendations require authorized, grounded context.
Candidate + Impact →DAIL	Verification must test an exact proposed change and its impact.
DAIL →Promotion	Promotion cannot bypass verification.
Promotion →Trusted State	Trusted state requires authoritative post-change confirmation.
Observability →Production	Production release requires measurable health and evidence.
Testing →Every Gate	Each phase must prove its contract before dependent work becomes authoritative.

7. MVP Vertical Slice Strategy

The roadmap should produce progressively stronger vertical slices rather than waiting until the end for the first integrated system.

Slice	Scope	Definition of success
VS-1	Tenant →API →DB →audit	Authorized user can create/read a tenant-scoped resource with traceable audit.
VS-2	AWS connection →discovery →asset	Controlled AWS account can be connected and discovered into canonical assets.
VS-3	Asset →observation →security finding	A known AWS configuration produces a deterministic, evidenced finding.
VS-4	Finding →risk →dependency/impact	Risk and candidate impact are computed from versioned inputs.
VS-5	AI recommendation →validated candidate	LLM output is structured, grounded, validated, and cannot directly mutate cloud state.
VS-6	Candidate →DAIL verification	Exact candidate version receives deterministic verification and evidence.
VS-7	Approval →promotion →reconciliation	Authorized change executes, outcome is reconciled, and trusted state advances only when valid.
VS-8	Drift →invalidation →re-analysis	Post-promotion drift invalidates affected trusted state and triggers appropriate analysis.

VS-7 is the MVP control-loop milestone. VS-8 demonstrates that the system is not merely a one-time deployment tool but a continuously reconciling security/infrastructure intelligence platform.

8. Detailed Backend Build Order

1. Project bootstrap, configuration, dependency management, coding standards.
2. Tenant/organization/scope model.
3. Identity, authentication integration boundary, RBAC/authorization middleware.
4. Stable identifiers and domain error model.
5. Database schema, migrations, repository/data-access conventions.
6. Audit event and evidence metadata foundation.
7. Object-storage abstraction.
8. Durable job/queue abstraction.
9. API framework, request validation, error envelopes, pagination, idempotency.
10. Observability middleware and correlation/tracing.
11. AWS provider interface and mock provider.
12. CloudConnection and STS role-assumption flow.
13. Discovery partitioning/checkpointing.
14. Asset identity/reconciliation and observations.
15. Configuration snapshots/diffs and freshness.
16. Security rules/findings/risk.
17. Dependency graph and impact analysis.

18. AI gateway/context/structured output/guardrails.
19. Remediation candidate/version lifecycle.
20. DAIL verification engine.
21. Approval/policy/promotion orchestration.
22. AWS mutation execution and reconciliation.
23. Trusted state/drift detection.
24. Reporting and external integrations.
25. Performance, resilience, DR, and production hardening.

9. Frontend / UX Build Order

26. Application shell, authentication boundary, navigation, tenant/scope selector.
27. Overview/dashboard with system health and current risk.
28. Cloud connections and discovery jobs.
29. Asset inventory with search/filter/pagination and freshness.
30. Finding list/detail and risk views.
31. Dependency graph and impact analysis views.
32. AI explanation/recommendation experience with explicit confidence/grounding/degraded states.
33. Remediation candidate and version timeline.
34. DAIL verification status/evidence view.
35. Approval/governance and promotion workflow.
36. Promotion outcome, reconciliation, trusted-state and drift views.
37. Evidence/audit explorer and reports.
38. Jobs/notifications/error/empty/partial/stale states.
39. Accessibility, responsive behavior, performance and UX regression.

10. API Implementation Sequence

Group	Representative capabilities	Gate
Identity	Session/auth boundary, roles, authorization	Tenant isolation
Cloud	Connections, scopes, discovery jobs	Discovery correctness
Inventory	Assets, observations, snapshots, diffs	Identity reconciliation
Security	Rules, findings, risk	Deterministic analysis
Dependency	Edges, graph queries, impact	Graph correctness
AI	Context, requests, recommendations	Safety/structured validation
Candidates	Candidate/version lifecycle	Immutable version semantics
DAIL	Verification runs/results	Exact-version binding
Governance	Approvals, policy decisions	Authorized decision path
Promotion	Preflight, execution, reconciliation	Trusted-state safety
Evidence	Evidence artifacts, audit events	Durable proof
Operations	Jobs, reports, health, metrics	Operational readiness
Integrations	Webhooks/events/external sync	Isolation/idempotency

11. Data & Persistence Implementation Strategy

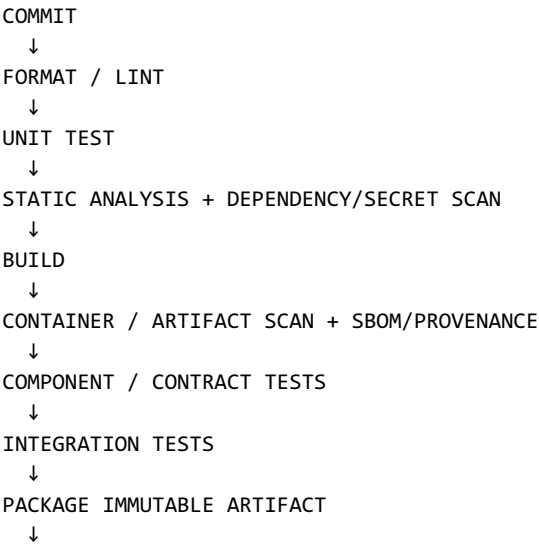
- Create schema from Specification 05 and lifecycle/state constraints from Specification 06.
- Use explicit version columns or equivalent optimistic concurrency controls.
- Keep lifecycle transition logic in domain/application code, not only in UI.
- Use database transactions for state changes that must be atomic.
- Never hold a DB transaction open while waiting on AWS, LLM, or external integrations.
- Store large evidence/artifacts in object storage and durable metadata in the relational store.
- Use retention policies and archival for high-volume audit/event data.
- Build migration tests and rollback/recovery procedures before production deployment.
- Introduce derived/read-optimized representations only after canonical semantics are stable.

12. Infrastructure & Environment Roadmap

Stage	Infrastructure deliverables
Local	Docker/local dependencies, mock AWS provider, mock LLM, local object storage/DB/queue equivalents where practical
Dev	AWS networking, compute, DB, queue, object storage, secrets/KMS, observability, Terraform state
Test	Isolated test environment + controlled AWS integration account
Staging	Production-like topology, realistic scale tests, DR/rollback tests
Prod	Hardened AWS topology, WAF/edge, backups, alerting, least privilege, deployment controls
Recovery	Terraform rebuild path, backup restore, queue recovery, secrets/identity recovery, reconciliation procedures

Environment promotion must use immutable artifacts and the same core security/lifecycle controls as production.

13. CI/CD Implementation Pipeline



```

DEPLOY TEST
↓
SYSTEM / E2E / SECURITY TESTS
↓
PERFORMANCE REGRESSION CHECKS
↓
STAGING DEPLOY
↓
SMOKE + RELEASE VERIFICATION
↓
GOVERNED PRODUCTION DEPLOY
↓
POST-DEPLOY HEALTH / RECONCILIATION
↓
RELEASE EVIDENCE

```

- Use federated/OIDC CI identity where supported; avoid long-lived cloud credentials.
- Artifact digests, deployment IDs, commit IDs, and environment IDs must be traceable.
- Database migrations must be compatible with the deployed application transition strategy.
- Rollback must be deterministic where possible and followed by state reconciliation.
- Deployment automation cannot bypass CloudShield authorization or governed cloud mutation controls.

14. Testing Strategy by Phase

Phase	Mandatory tests before gate
P0	CI smoke, repository validation, security baseline
P1	Unit, repository, authorization, tenant-isolation, migration, API contract
P2	AWS adapter contract, discovery integration, identity reconciliation, pagination/throttling
P3	Rule correctness, finding lifecycle, risk regression, evidence
P4	Graph correctness, identity conflict, impact traversal, performance
P5	Structured output, prompt injection, grounding, model/provider failure, AI regression
P6	DAIL invariants, exact-version binding, evidence, negative/failure paths
P7	Promotion authorization, duplicate prevention, ambiguous mutation reconciliation, trusted-state rules
P8	E2E, accessibility, UX states, integration contracts
P9	Load, stress, soak, fault injection, DR, security operations, rollback
P10	Full regression, release candidate E2E, production smoke, readiness evidence

15. Engineering Gates

Gate G0 — Build-Ready

Specifications baselined; repository and architecture conventions established; no unresolved blocking architecture ambiguity.

Gate G1 — Platform-Ready

Tenant isolation, authorization, persistence, jobs, audit, observability and CI are functioning.

Gate G2 — Discovery-Ready

AWS connection/discovery produces stable, provenance-aware assets and observations.

Gate G3 — Analysis-Ready

Security findings and risk are deterministic, evidenced and testable.

Gate G4 — Impact-Ready

Dependency graph and candidate-bound impact analysis are correct and bounded.

Gate G5 — AI-Ready

AI is provider-neutral, structured, grounded, guarded and non-authoritative.

Gate G6 — Verification-Ready

DAIL can verify exact candidate versions and produce durable evidence.

Gate G7 — Control-Loop-Ready

Governed promotion, AWS execution, reconciliation and trusted-state transitions work end-to-end.

Gate G8 — Product-Ready

UX, jobs, evidence, reporting and selected integrations meet acceptance criteria.

Gate G9 — Production-Ready

Security, performance, reliability, DR, rollback, observability and operations are proven.

Gate G10 — MVP-Accepted

End-to-end MVP criteria passed, critical risks closed/accepted, production release evidence complete.

16. MVP Scope

Capability	MVP decision
AWS	Primary provider; controlled account/region/service coverage sufficient for core demonstration and production baseline
Cloud onboarding	Required
Asset inventory	Required
Configuration observations/snapshots	Required
Security findings	Required
Risk scoring	Required

Dependency graph	Required for supported resource relationships
Impact analysis	Required for remediation candidates
AI explanation/recommendation	Required through hosted model/provider abstraction
Local foundation model	Not required
Remediation candidates	Required
DAIL verification	Required
Human approval/governance	Required for governed production-affecting changes
Cloud mutation/promotion	Required only for explicitly supported, least-privilege operations
Trusted state	Required
Drift/invalidation	Required baseline
Evidence/audit	Required
Observability	Required
Terraform/AWS deployment	Required
External integrations	Selected high-value integration(s); broad ecosystem is later
Multi-cloud	Architecture-ready, not broad MVP implementation
Autonomous unrestricted remediation	Out of scope

17. Priority Classification

Priority	Meaning	Examples
P0 Critical	Required for safety, correctness, or core MVP control loop	Tenant isolation, auth, lifecycle, DAIL, evidence, reconciliation
P1 Core	Required for useful MVP functionality	AWS discovery, findings, risk, dependency, AI, candidate workflow
P2 Product	Required for polished operational product	Advanced UX, reporting, integrations, richer dashboards
P3 Expansion	Post-MVP capability	Multi-cloud breadth, advanced graph infrastructure, local model optimization
P4 Research	Experimental/non-core	Advanced autonomous reasoning, new verification research, experimental model strategies

18. Parallel Work Strategy

After P1 foundations are stable, teams can work in parallel while preserving contract boundaries.

Parallel stream	Can begin after	Must consume
AWS discovery	P1	Identity, persistence, provider interface
Security/risk	Early P1 + domain schema	Observation contract

Dependency	P2 identity contract	Asset identity + observations
AI	P1 domain/API contracts	Authorized context, candidate schema
Frontend	P1 API conventions; progressively after each domain slice	Stable API contracts
DAIL	Candidate/verification contract design can begin early; implementation after relevant domain contracts	Candidate version, impact, evidence
Terraform/DevOps	P0/P1	Architecture and deployment contracts
Testing	P0 onward	Every implementation contract
Performance	Instrumentation from P1; optimization after P2/P3 representative workload	Telemetry + benchmark harness
Security operations/DR	Architecture from P1; full validation after P9	Observability + infrastructure baseline

19. Master Implementation Risks

Risk	Impact	Mitigation
Premature microservices	Complexity/latency/operational burden	Keep modular architecture; split only on measured need
AI overreach	Unsafe cloud changes	Strict AI abstraction, structured output, guardrails, DAIL, governance
Weak tenant isolation	Security breach	Tenant-scoped authorization at every layer + adversarial tests
Unstable cloud identity	Incorrect findings/impact	Provider identifiers + deterministic reconciliation + conflict state
False bounded impact	Unsafe remediation	Unknown/partial graph state remains explicit; fail closed for critical gates
Provider throttling	Discovery delays	Partitioning, checkpoints, backoff, quota-aware scheduling
LLM outage	Feature degradation	Provider abstraction, fallback/degraded behavior, no authority
Ambiguous cloud mutation	Duplicate/destructive change	Idempotency where possible + reconciliation before retry
Evidence gaps	Untrustworthy system	Durable evidence/audit before dependent state transitions
Schema migration failure	Downtime/data corruption	Expand/contract, tested migrations, backups, rollback
Noisy tenant	SLO breach	Quotas, bulkheads, fair scheduling
Insufficient benchmarks	Bad capacity decisions	Representative datasets + reproducible harness
Scope creep	Delayed MVP	P0/P1/P2/P3 classification and gate

		discipline
Foundation-model distraction	Lost engineering focus	Use existing LLM; local model is future adapter
Automation bypass	Control failure	All automation passes the same lifecycle/policy/verification gates

20. Implementation Readiness Checklist

- Specifications 00–38 are available and treated as the current engineering baseline.
- Repository and branch strategy are defined.
- Architecture Decision Record process exists.
- Development, test, staging and production environments are defined.
- Terraform structure and remote state strategy are established.
- CI/CD security gates are operational.
- Domain model and lifecycle state machine are translated into code-level contracts.
- Tenant isolation and authorization test strategy is implemented.
- AWS mock/provider adapter exists for local testing.
- LLM mock/provider abstraction exists for deterministic tests.
- Durable job abstraction exists.
- Observability and correlation IDs exist before large-scale async work is built.
- Evidence/audit model exists before promotion functionality is implemented.
- Test data and representative workload datasets are available.
- Acceptance criteria for the first vertical slice are executable.

21. Feature-Level Definition of Done

40. Requirement traced to one or more specifications.
41. Domain behavior defined and lifecycle impact identified.
42. Authorization and tenant-isolation behavior implemented.
43. API/service contract defined where applicable.
44. Persistence and migration changes implemented and tested.
45. Unit and component tests pass.
46. Integration/contract tests pass for external boundaries.
47. Observability, audit, and evidence requirements implemented.
48. Failure, timeout, retry, idempotency, and reconciliation behavior defined.
49. Security threat considerations addressed.
50. Performance impact measured or explicitly bounded.
51. Documentation and API/schema changes updated.
52. CI security/quality gates pass.
53. Feature demonstrated through an appropriate vertical slice.
54. Acceptance criteria passed with evidence.

22. MVP Release Definition of Done

- All P0 critical requirements are implemented and tested.
- Core vertical slice VS-1 through VS-8 is functional for the supported AWS scope.
- Tenant isolation has passed adversarial testing.

- Cloud discovery produces stable identities and provenance-aware observations.
- Security/risk findings are deterministic and evidenced.
- Dependency and impact analysis are version-bound and safe under incomplete data.
- AI outputs are structured, grounded, guarded, evaluated, and non-authoritative.
- DAIL verification is exact-version-bound and cannot be bypassed.
- Governance/approval is explicit for governed production changes.
- Cloud mutation uses least privilege, preflight validation, and reconciliation.
- Trusted state is earned only after authoritative confirmation.
- Drift invalidation works for supported resources.
- Audit/evidence durability meets MVP baseline.
- Performance and reliability targets from Specification 38 are validated or explicitly accepted through governance.
- DR targets from Specification 33 are tested.
- CI/CD, Terraform, rollback, and production deployment are operational.
- Security operations and incident runbooks exist.
- Production dashboards and alerts exist.
- No critical unresolved security or correctness defect remains without explicit risk acceptance.

23. Roadmap Execution Metrics

Metric	Purpose
Specification coverage	Percentage of implementation work linked to 00–38
Gate pass rate	How often phase gates pass without rework
Critical defect count	Open P0 correctness/security issues
E2E control-loop pass rate	Health of the complete product path
Deployment success rate	Release reliability
Rollback/recovery success	Operational resilience
SLO attainment	Production performance/reliability
Reconciliation backlog	Unresolved external ambiguity
Verification failure rate	DAIL quality/safety signal
AI grounded-correct response rate	AI usefulness/safety signal
Tenant-isolation test pass rate	Security confidence
Evidence completeness	Trust/audit readiness
Mean time to recovery	Operational resilience
Queue age / saturation	Async capacity
Cost per useful workload	Efficiency

24. Traceability to Specification Suite

Spec	Specification	Roadmap dependency	Primary phase
00	Master System Specification	Product boundary, control loop, trust model	All phases
01	SRS	Functional/non-functional requirements	All phases

02	System Architecture	Containers, workers, data, boundaries	P0–P10
03	Functional Specification	Behavior and lifecycle workflows	P1–P8
04	Product & UX	User workflows and interface acceptance	P2–P10
05	Domain & Data Model	Canonical entities and persistence	P1–P8
06	State & Lifecycle	Transitions, invalidation, reconciliation	P1–P10
07	Identity & Dependency	Stable identity and graph semantics	P2–P7
08	Security & Threat Model	Threats, trust boundaries, controls	All phases
09	Intelligence & AI Architecture	AI abstraction and context architecture	P5
10	LLM Integration	Provider adapters, reliability, telemetry	P5
11	AI Safety & Guardrails	AI safety and authority boundaries	P5–P7
12–26	DAIL subsystem package	Verification, evidence, experiments, DevOps	P6–P9
27	AWS & Terraform	Infrastructure implementation	P0–P10
28	API Specification	External/internal API contracts	P1–P8
29	Cloud Integration	AWS integration and reconciliation	P2–P7
30	External Integration	External system contracts	P8–P10
31	Observability & Logging	Telemetry, SLOs, audit/evidence	P1–P10
32	Security Operations	Detection and response	P9–P10
33	Disaster Recovery	RTO/RPO and restore/reconciliation	P9–P10
34	Deployment & DevOps	CI/CD, release, rollback	P0–P10
35	System Testing	Test strategy and gates	All phases
36	DAIL Experiment	Controlled verification experiments	P6–P9
37	AI Evaluation & Benchmark	AI quality/regression/cost/safety	P5–P10
38	Performance & Reliability	SLOs, capacity, resilience, benchmarks	P1–P10

25. Suggested Engineering Ownership Model

Role / team	Primary ownership
-------------	-------------------

Product/Technical Lead	Scope, architecture governance, gate decisions, cross-spec traceability
Backend Platform	P1 foundation, API, lifecycle, persistence
Cloud/Infrastructure	AWS integration, Terraform, deployment infrastructure
Security Engineering	Threat controls, tenant isolation, security analysis, SecOps
AI/ML Engineering	AI abstraction, context, evaluation, guardrails
DAIL Engineering	Verification, invariants, evidence, promotion readiness
Frontend/Product Engineering	UX and product workflows
SRE/Platform Operations	Observability, SLOs, DR, capacity, incident readiness
QA/Systems Engineering	Integration, E2E, performance, resilience, regression
Data/Analytics	Risk/telemetry/reporting/evaluation datasets where applicable

Ownership may be combined for a small team; the responsibilities should not disappear merely because roles are combined.

26. Exact Implementation Command Sequence

55. Freeze specification baseline 00–38 and create an implementation branch/repository baseline.
56. Create ADR-001 for the MVP architecture and explicitly record modular-monolith/small-worker decision.
57. Create repository structure, tooling, CI, security scanning, local development environment, and coding conventions.
58. Implement tenant, identity, authorization, stable IDs, domain errors, and audit foundations.
59. Implement PostgreSQL schema/migrations, repositories, object-storage abstraction, and durable job model.
60. Implement API skeleton and observability middleware.
61. Implement AWS provider abstraction and mock provider.
62. Implement CloudConnection, secure role assumption, discovery jobs, Asset, Observation, Snapshot, and reconciliation.
63. Implement first deterministic security rule and complete VS-3.
64. Expand security/risk baseline and complete analysis gate.
65. Implement canonical dependency graph and first impact scenario.
66. Implement AI abstraction, one hosted provider adapter, structured response validation, guardrails, and AI evaluation baseline.
67. Implement RemediationCandidate and CandidateVersion.
68. Implement DAIL verification engine for the first supported remediation type.
69. Implement approval/policy decision and promotion preflight.
70. Implement one safe AWS mutation with least privilege and full reconciliation.
71. Implement trusted-state transition and drift invalidation.
72. Complete the corresponding frontend golden path.
73. Run full E2E control-loop tests and harden failure paths.
74. Execute load/stress/soak/fault-injection/DR/security tests.
75. Fix critical findings, establish production dashboards/runbooks, and run release candidate.
76. Deploy through governed CI/CD and validate post-deployment reconciliation.
77. Declare MVP only after Gate G10.

27. Recommended First Governed Remediation

The first cloud mutation should be deliberately narrow, reversible where possible, easy to verify, and rich in deterministic evidence. It should exercise the complete control loop without requiring broad autonomous permissions.

- Choose one well-understood AWS configuration remediation within the supported discovery/security scope.
- Require deterministic preconditions and a small blast radius.
- Represent the change as a CandidateVersion rather than executing an AI-generated command directly.
- Require dependency/impact evaluation where applicable.
- Require DAIL verification before approval/promotion.
- Use a dedicated least-privilege promotion identity.
- Capture before-state, intended change, execution result, after-state, and reconciliation evidence.
- Make failure and ambiguous outcomes explicit and recoverable.

The exact first mutation should be selected during implementation based on the supported AWS security use case; this roadmap intentionally does not invent a specific production mutation.

28. Stop / Block Conditions

- Tenant isolation or authorization defect discovered.
- Critical lifecycle invariant can be bypassed.
- DAIL verification can be bypassed or bound to mutable/unidentified content.
- Promotion can occur without required governance or evidence.
- Ambiguous cloud mutation can be blindly retried.
- Trusted state can be created without authoritative confirmation.
- Unknown/partial/stale state can be represented as clean.
- Critical audit/evidence durability is below required baseline.
- Production deployment can bypass required security/quality gates.
- AI output can directly acquire cloud mutation authority.
- Critical DR/reconciliation path is untested or known unsafe.
- Performance optimization requires weakening a safety or trust invariant.

A stop condition blocks advancement until fixed or explicitly accepted by the defined engineering/security governance authority.

29. Post-MVP Evolution

- Expand AWS service/resource coverage based on customer value and telemetry.
- Add Azure/GCP adapters behind the same provider-neutral cloud integration contract.
- Introduce graph-optimized derived storage only when measured workloads justify it.
- Add local/open-weight model adapters when privacy, cost, latency, or deployment requirements justify them.
- Consider model fine-tuning only after sufficient evaluated proprietary task data exists.
- Expand external integrations and ITSM/source-control workflows.
- Increase automation only while preserving the same verification/governance/trusted-state architecture.
- Introduce more advanced deployment strategies such as canarying where operational scale warrants them.

30. Master Roadmap Acceptance Criteria

AC-01: Every implementation phase has explicit deliverables and a measurable gate.

AC-02: The roadmap covers the entire CloudShield AI product and treats DAIL as a bounded subsystem.

AC-03: Specifications 00–38 are traceable to implementation phases and workstreams.

AC-04: The roadmap contains a complete MVP vertical slice from discovery through trusted state.

AC-05: AI remains non-authoritative throughout implementation.

AC-06: DAIL verification is mandatory for the supported governed remediation path.

AC-07: Tenant isolation and least-privilege identity are foundational, not post-MVP work.

AC-08: Evidence and audit are implemented before promotion functionality.

AC-09: AWS mutation includes preflight checks, least privilege, idempotency/reconciliation, and authoritative confirmation.

AC-10: Unknown, partial, stale, or ambiguous state cannot be converted into trusted/clean state to satisfy roadmap milestones.

AC-11: Performance, resilience, security, and DR testing are release gates.

AC-12: CI/CD and Terraform are part of implementation, not a final deployment afterthought.

AC-13: Local foundation-model training is not required for MVP.

AC-14: The roadmap provides a clear path from local development to production deployment.

AC-15: MVP release requires explicit evidence-based Gate G10 acceptance.

31. Master Definition of Done

- All required MVP specifications have corresponding implementation coverage.
- Core domain model, lifecycle, identity, dependency, security, AI, DAIL, governance, evidence, and trusted-state semantics are implemented.
- The AWS discovery →analysis →candidate →impact →DAIL →approval →promotion →reconciliation →trusted-state path works end-to-end.
- All critical authorization and tenant-isolation paths are tested.
- AI provider abstraction and safety guardrails are active.
- DAIL promotion authority is isolated from ordinary application and AI workloads.
- Observability, audit, and evidence are operational.
- Performance and reliability targets have been measured under representative workload.
- Security, fault-injection, DR, rollback, and reconciliation tests have passed.
- Terraform and CI/CD produce reproducible, traceable deployments.
- Production runbooks and incident/recovery procedures exist.
- Critical defects are closed or explicitly risk-accepted.
- Release evidence demonstrates compliance with the roadmap gates.
- Gate G10 has been formally accepted.

CloudShield AI — Master Implementation Roadmap v1.0

Build the control loop. Prove every gate. Earn trusted state.